

Lecture 4/Module 2

CS3212 Algorithms

Aya Zirikly
09/03/2026

MergeSort —Key Properties & Trade-Offs

-  **Stability Guaranteed:** Maintains the relative ordering of equal key elements, critical for multi-key database sorting.
-  **Predictable Performance:** Worst, average, and best-case time complexities are all strictly $O(n \log n)$.
-  **$O(n)$ Extra Space:** Requires auxiliary arrays during the merge step, unlike in-place algorithms like QuickSort.
-  **Optimal for Linked Lists:** Can sort linked lists in $O(n \log n)$ time with $O(1)$ extra pointer manipulations.

Agenda & Learning Goals

- ❑ **More $T(n)$ examples**
- ❑ **Master Theorem Shortcut**
- ❑ **QuickSort Deep Dive:** Partitioning subroutines, best vs. worst vs. average recursion tree shapes.
- ❑ **Selection Algorithms:** QuickSelect and Deterministic Linear-Time Selection (Median-of-Medians).
- ❑ **Theoretical Lower Bound:** Prove why no comparison-based sort can beat $\Omega(n \log n)$ using Decision Trees.

Root-Dominated Tree Example (CLRS)

Recurrence: $T(n) = 3 * T(n/4) + c * n^2$

Recurrence: $T(n) = 3 * T(n/4) + c * n^2$

Level 0 (Root): 1 node of size $n \rightarrow$ Work = $c * n^2$

$3 * T(n/4)$ passed down to next level
Recursive call

Level 1: 3 nodes of size $n/4 \rightarrow$ Work = $3 * c * (n/4)^2 = (3/16) * c * n^2$

Level 2: 9 nodes of size $n/16 \rightarrow$ Work = $9 * c * (n/16)^2 = (9/256) * c * n^2$

Level i: 3^i nodes of size $n/(4^i) \rightarrow$ Work = $(3/16)^i * c * n^2$

Summing Across All Levels:

- Total Work = $c * n^2 * \sum_{i=0}^h (3/16)^i$
- Since $(3/16) < 1$, this forms a **decreasing geometric series** bounded by a constant factor $1 / (1 - 3/16) = 16/13$.
- **Conclusion:** The root node completely dominates total runtime $\rightarrow T(n) = \Theta(n^2)$.

Exact Operations vs. Asymptotic Bounds (Sedgewick & Wayne)

Why Constant Factors Matter in Real-World Systems

Big-O notation hides constant factors, but exact operation counts explain actual execution speed on modern hardware:

Exact Operation Counts for Standard Mergesort:

- **Array Accesses:** Exactly $6 * N * \log_2(N)$ array accesses to sort an array of size N .
- **Element Compares:** Between $0.5 * N * \log_2(N)$ and $1.0 * N * \log_2(N)$ comparisons.
- **Auxiliary Memory:** Allocates extra memory equal to $1.0 * N$.

Key Takeaway: While Mergesort achieves optimal $\Theta(N \log N)$ theoretical scaling, its requirement of 6 array accesses per level makes it slower in cache memory than Quicksort in practice.

Exact Operations vs. Asymptotic Bounds (Sedgewick & Wayne)

		Array Accesses
1. Copy to Auxiliary Array	Read from <code>main[i]</code> , Write to <code>temp[i]</code>	2 (1 Read + 1 Write)
2. Compare & Read for Merge	Read element from <code>temp[i]</code> to decide where it goes	2 (1 Read from Left sub-array + 1 Read from Right sub-array during comparison)
3. Write Sorted Back	Write the winning element back into <code>main[i]</code>	2 (1 Write back to main + 1 supplementary update/pointer access)

Exact Op

In Merge Sort, to merge two sorted lists of size $N/2$ into a single list of size N you compare elements one by one.

(Wayne)

Why Constant F

Big-O notation hi
modern hardware

- In the **worst case**, you compare elements until one list is almost entirely empty. Merging two sublists of size $N/2$ takes at most **$N - 1$ comparisons**.
- A recursion tree for array size N has approximately **$\log_2(N)$ levels**.
- At every level of the tree, all sublists combined require at most N comparisons.

ed on

Exact Operation

Multiply the work per level by the number of levels:

total Comparisons (Worst Case)} $\sim N \log_2(N) = 1.0 * N \log_2(N)$

- **Array Accesses:** Exactly $0.5 * N * \log_2(N)$ array accesses to sort an array of size N .
- **Element Compares:** Between $0.5 * N * \log_2(N)$ and $1.0 * N * \log_2(N)$ comparisons.
- **Auxiliary Memory:** Allocates extra memory equal to $1.0 * N$.

Key Takeaway:

of 6 array acces

The **best case** happens when all elements in one sublist are strictly smaller than all elements in the second sublist (for instance, merging $[1, 2, 3]$ and $[4, 5, 6]$). You compare the elements until the entire first sublist is exhausted.

Since the first sublist only has **$N/2$ elements**, you only perform **$N/2$ comparisons** (or $0.5 N$). Once that sublist runs out, you simply append the remaining half without any further comparisons.

Across $\log_2(N)$ levels, doing $0.5 N$ comparisons per level

requirement

The Master Theorem

Applicable Form: $T(n) = a * T(n/b) + f(n)$

- $a \geq 1$: Number of recursive subproblems created at each step.
- $b > 1$: Factor by which the input size is divided.
- $f(n)$: Work done outside recursive calls (Divide + Combine cost).

Core Intuition: The Master Theorem compares the rate of work created at the leaves $n^{(\log_b a)}$ against the work done at the root $f(n)$.

RECURSION TREE FOR $T(n) = a * T(n/b) + f(n)$

Level	Tree Structure	Work Per Level
Level 0 (Root)	$ \begin{array}{c} [f(n)] \\ / \quad \quad \backslash \\ / \quad \quad \backslash \quad (a \text{ branches}) \\ / \quad \quad \backslash \end{array} $	$f(n)$
Level 1	$ \begin{array}{c} [f(n/b)] \quad [f(n/b)] \quad \dots \quad [f(n/b)] \\ / \quad \backslash \quad / \quad \backslash \quad / \quad \backslash \end{array} $	$a * f(n/b)$
Level 2	$ \begin{array}{c} [f(n/b^2)] \quad \dots \quad [f(n/b^2)] \quad \dots \quad [f(n/b^2)] \\ \quad \quad \quad \quad \quad \quad \\ : \quad \quad \quad : \quad \quad \quad : \\ \quad \quad \quad \quad \quad \quad \end{array} $	$a^2 * f(n/b^2)$
Level h (Leaves)	$ \begin{array}{c} o \quad o \quad o \quad o \quad \dots \quad o \quad o \quad o \quad o \\ \backslash \text{-----} / \\ \text{Total Leaves} = a^{(\log_b n)} = n^{(\log_b a)} \end{array} $	$\Theta(n^{(\log_b a)})$

Key Structural Metrics:

- * Branching factor (a): Number of child subproblems created per node.
- * Reduction ratio (b): Factor by which the input size shrinks per level.
- * Tree Height (h): $h = \log_b(n)$ levels from root to base cases.
- * Total Leaves: $a^h = a^{(\log_b n)} = n^{(\log_b a)}$ leaf nodes.

RECURSION TREE FOR $T(n) = a * T(n/b) + f(n)$

Level	Tree Structure	Work Per Level
Level 0 (Root)	$ \begin{array}{c} [f(n)] \\ / \quad \quad \backslash \\ / \quad \quad \backslash \quad (a \text{ branches}) \\ / \quad \quad \backslash \end{array} $	$f(n)$
Level 1	$ \begin{array}{c} [f(n/b)] \quad [f(n/b)] \quad \dots \quad [f(n/b)] \\ / \quad \backslash \quad / \quad \backslash \quad / \quad \backslash \end{array} $	$a * f(n/b)$
Level 2	$ \begin{array}{c} [f(n/b^2)] \quad \dots \quad [f(n/b^2)] \quad \dots \quad [f(n/b^2)] \\ \quad \quad \quad \quad \quad \quad \\ : \quad \quad \quad : \quad \quad \quad : \\ \quad \quad \quad \quad \quad \quad \end{array} $	$a^2 * f(n/b^2)$
Level h (Leaves)	$ \begin{array}{c} o \quad o \quad o \quad o \quad \dots \quad o \quad o \quad o \quad o \\ \backslash \text{-----} / \\ \text{Total Leaves} = a^{(\log_b n)} = n^{(\log_b a)} \end{array} $	$\Theta(n^{(\log_b a)})$

**Power-Base Swap
Property of logarithms**

Key Structural Metrics:

- * Branching factor (a): Number of child subproblems created per node.
- * Reduction ratio (b): Factor by which the input size shrinks per level.
- * Tree Height (h): $h = \log_b(n)$ levels from root to base cases.
- * Total Leaves: $a^h = a^{(\log_b n)} = n^{(\log_b a)}$ leaf nodes.

RECURSION TREE FOR $T(n) = a * T(n/b) + f(n)$

Level	Tree Structure	Work Per Level
Level 0 (Root)	$ \begin{array}{c} [f(n)] \\ / \quad \quad \backslash \\ / \quad \quad \backslash \quad (a \text{ branches}) \\ / \quad \quad \backslash \end{array} $	$f(n)$
Level 1	$ \begin{array}{c} [f(n/b)] \quad [f(n/b)] \quad \dots \quad [f(n/b)] \\ / \quad \backslash \quad / \quad \backslash \quad / \quad \backslash \end{array} $	$a * f(n/b)$
Level 2	$ \begin{array}{c} [f(n/b^2)] \quad \dots \quad [f(n/b^2)] \quad \dots \quad [f(n/b^2)] \\ \quad \quad \quad \quad \quad \quad \\ : \quad \quad \quad : \quad \quad \quad : \\ \quad \quad \quad \quad \quad \quad \end{array} $	$a^2 * f(n/b^2)$
Level h (Leaves)	$ \begin{array}{c} o \quad o \quad o \quad o \quad \dots \quad o \quad o \quad o \quad o \\ \backslash \text{-----} / \\ \text{Total Leaves} = a^{(\log_b n)} = n^{(\log_b a)} \end{array} $	$\Theta(n^{(\log_b a)})$

Key Structural Metrics:

- * Branching factor (a): Number of child subproblems created per node.
- * Reduction ratio (b): Factor by which the input size shrinks per level.
- * Tree Height (h): $h = \log_b(n)$ levels from root to base cases.
- * Total Leaves: $a^h = a^{(\log_b n)} = n^{(\log_b a)}$ leaf nodes.

The 3 Master Theorem Cases

Case 1 (Leaf-Dominated Work):

- Condition: $f(n) = O(n^{\log_b(a) - \epsilon})$ for some constant $\epsilon > 0$.
- Meaning: Non-recursive work grows slower than leaf expansion. Leaves dominate total runtime.
- Result: $T(n) = \Theta(n^{\log_b a})$

Case 1 (Leaf-Dominated Work)

$$f(n) = O(n^{(\log_b(a) - \epsilon)}) \text{ for some constant } \epsilon > 0.$$

What is ϵ (epsilon ϵ)? any **positive constant** $\epsilon > 0$, such as 0.1, 0.5, 1, or 2

In asymptotic notation, ϵ enforces a **polynomial gap** between two functions:

- If $f(n) = O(n^{\log_b a - \epsilon})$, it means $f(n)$ is not just smaller than $n^{\log_b a}$, but smaller by a factor of at least n^ϵ :

$$f(n) \leq C \cdot \frac{n^{\log_b a}}{n^\epsilon}$$

This shows that $f(n)$ isn't just smaller than $n^{\log_b a}$; it is smaller by a factor of at least n^ϵ (a full polynomial factor like $n^{0.1}$, $n^{0.5}$, or n^1).

Case 1 (Leaf-Dominated Work)

$$f(n) = O(n^{(\log_b(a) - \epsilon)}) \text{ for some constant } \epsilon$$

What is ϵ (epsilon ϵ)? any **positive constant** $\epsilon > 0$, such as 0.1, 0.5, 1, or 2

In asymptotic notation, ϵ enforces a **polynomial gap** between two functions:

Non-recursive work grows
slower than leaf expansion
**=> Leaves dominate total
runtime**

- If $f(n) = O(n^{\log_b a - \epsilon})$, it means $f(n)$ is not just smaller than $n^{\log_b a}$, but smaller by a factor of at least n^ϵ :

$$f(n) \leq C \cdot \frac{n^{\log_b a}}{n^\epsilon}$$

This rules out functions that are only slightly smaller by logarithmic factors (like $f(n) = \frac{n^{\log_b a}}{\log n}$), ensuring a true polynomial difference in growth rate.

Leaves dominate total runtime

Calculate Total Work at the Leaves

- At tree height $h = \log_b n$, the total number of leaf nodes is $a^{(\log_b n)} = n^{(\log_b a)}$.
- Assuming each leaf represents a base case taking $O(1)$ work, the total work done by all leaves combined is $\Theta(n^{(\log_b a)})$.

Compare Root Work to Leaf Work

- ❑ The non-recursive work performed at Level 0 (the root node) is $f(n)$.
- ❑ Substituting the Case 1 condition: $f(n) = O(n^{(\log_b(a) - \epsilon)}) \Rightarrow$

$$\text{Root Work } f(n) = O(\text{Leaf Work} / n^\epsilon)$$

- ❑ This relationship explicitly shows that as n grows large, the work done at the root $f(n)$ becomes completely negligible compared to the total work done at the leaves.

Sum across levels

When summing the work across all levels of the tree from top to bottom:

- **Level 0 (Root):** $f(n)$
- **Level 1:** $a * f(n/b)$
- **Level 2:** $a^2 * f(n/b^2)$
- ...
- **Level h (Leaves):** $n^{(\log_b(a))}$

Because $f(n)$ is polynomially smaller than $n^{(\log_b(a))}$ by a factor of n^e , the amount of work done per level **increases exponentially** as you move down the tree.

In mathematics, when a geometric series increases exponentially per term, the sum of the entire series is asymptotically equal to its **largest term**—which is the final level (the leaves).

Total Work $T(n)$ = Sum of All Levels = $\Theta(\text{Leaf Work}) = \Theta(n^{(\log_b(a))})$

The 3 Master Theorem Cases

Case 1 (Leaf-Dominated Work):

- Condition: $f(n) = O(n^{\log_b(a) - \epsilon})$ for some constant $\epsilon > 0$.
- Non-recursive work grows slower than leaf expansion. Leaves dominate total runtime.
- Result: $T(n) = \Theta(n^{\log_b a})$

The 3 Master Theorem Cases

Case 2 (Balanced Work Across Levels):

- Condition: $f(n) = \Theta(n^{\log_b a} * (\log n)^k)$ for $k \geq 0$.
- Meaning: Work is evenly distributed across tree levels.

Case 2 (Balanced Work Across Levels)

$$f(n) = \Theta(n^{\log_b a} (\log n)^k) \text{ for } k \geq 0$$

- $f(n)$: The non-recursive work performed at the root node.
- $n^{\log_b a}$: The total work capacity at the leaf level (number of base cases).
- $(\log n)^k$: A logarithmic multiplier ($k = 0, 1, 2, \dots$). When $k = 0$, $(\log n)^0 = 1$, meaning $f(n) = \Theta(n^{\log_b a})$.

This condition states that the non-recursive work at the root $f(n)$ matches the work capacity of the leaves $n^{\log_b a}$, up to a logarithmic factor.

The **logarithmic multiplier** $(\log n)^k$ is included in the Case 2 condition to account for divide-and-conquer algorithms where the non-recursive work $f(n)$ (the divide and combine steps) involves **extra logarithmic factors** beyond simple linear or polynomial work.

Where Does the Logarithm Come From

- ❖ In standard Merge Sort, merging two sorted halves of total size n takes linear time: $f(n) = \Theta(n)$. Here, $k = 0$, so $(\log n)^0 = 1$.
- ❖ In many real-world algorithms, the divide or combine step isn't just array scan—it involves logarithmic operations like searching, sorting, or tree lookups:
 - Merging via Binary Search: If an algorithm uses binary search during the combine phase, the work per level might be $f(n) = \Theta(n \log n)$. Here, $k = 1$.
 - Combining via Balanced Binary Search Trees: If combining subproblem results requires inserting n elements into a Red-Black Tree, the non-recursive work is $f(n) = \Theta(n \log n)$.

Level Work = $\Theta(n^{\log_b a} \cdot (\log n)^k)$

Sum Across All Levels = (Work per Level) * (Tree Height)

= $\Theta(n^{\log_b a} \cdot (\log n)^k) \cdot \Theta(\log n)$

= $\Theta(n^{\log_b a} \cdot (\log n)^{k+1})$

Recurrence Equation	Non-Recursive Work $f(n)$	Value of k	Work Per Level	Total Result $T(n)$
$T(n) = 2T(n/2) + n$	n	$k = 0$	$\Theta(n)$	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n \log n$	$n \log n$	$k = 1$	$\Theta(n \log n)$	$\Theta(n (\log n)^2)$
$T(n) = 2T(n/2) + n (\log n)^2$	$n (\log n)^2$	$k = 2$	$\Theta(n (\log n)^2)$	$\Theta(n (\log n)^3)$

Case 3 (Root-Dominated Work)

- Condition: $f(n) = \Omega(n^{(\log_b(a) + e)})$ AND regularity condition $a * f(n/b) \leq c * f(n)$ for $c < 1$.
- Meaning: Root work dwarfs subproblem work. Top level dominates total runtime.
- Result: $T(n) = \Theta(f(n))$

Master Theorem —Examples

Example 1: $T(n) = 4 * T(n/2) + n$

- Identify: $a = 4$, $b = 2$, $f(n) = n$.
- Compute leaf term: $n^{(\log_2 4)} = n^2$.
- Compare: $f(n) = n$ is polynomially smaller than n^2 (Case 1).
- Solution: $T(n) = \Theta(n^2)$.

Master Theorem —Examples

Example 2: $T(n) = 2 * T(n/2) + c * n$ (Merge Sort)

- ❖ Identify: $a = 2, b = 2, f(n) = c * n$.
- ❖ Compute leaf term: $n^{(\log_2 2)} = n^1 = n$.
- ❖ Comparing $f(n)$ to Leaf Capacity
 - Exact Polynomial Match: The work at the root, $f(n) = c * n$, grows at n^1 . If the leaves also total $n^{(\log_b a)} = n^1$, both parts grow at the exact same rate.
 - The combine step $c * n$ has no extra $\log n$ factor attached. Since $(\log n)^0 = 1$, we set $k = 0$.
- ❖ Every level of the tree does the exact same amount of work: $n^1 * (\log n)^0$.
- ❖ Total Complexity: Summing across all $\log n$ levels adds an extra logarithm, giving $\Theta(n^1 * (\log n)^{0+1}) = \Theta(n \log n)$.

Master Theorem —Examples

Example 3: $T(n) = 3 * T(n/4) + n^2$

- Identify: $a = 3$, $b = 4$, $f(n) = n^2$.
- Compute leaf term: $n^{(\log_4 3)} = n^{0.793}$.
- Compare: $f(n) = n^2$ is polynomially larger than $n^{0.793}$ (Case 3).
- Solution: $T(n) = \Theta(n^2)$.

When the Master Theorem Cannot Be Used

Standard Master Theorem Requirement: The recurrence MUST strictly match $T(n) = a * T(n/b) + f(n)$, where $a \geq 1$ is a fixed constant, $b > 1$ is a fixed constant, and all subproblems are equal in size.

- **Violation 1: Non-Constant Branching Factor (a) or Shrink Factor (b)**

- **Equation:** $T(n) = n * T(n/2) + n$

- **Failure:** Parameter $a = n$ (branching count grows with input size instead of being a constant number).

- **Equation:** $T(n) = 2 * T(\sqrt{n}) + n$

- **Failure:** Parameter b is not a constant divisor (input size shrinks by square root, not division by a constant b).

When the Master Theorem Cannot Be Used

- **Violation 3: Non-Polynomial Gap Between $f(n)$ and $n^{\log_b a}$**
 - **Equation:** $T(n) = 2 * T(n/2) + (n / \log n)$
 - Here $a = 2$, $b = 2$, so $n^{\log_b a} = n^1$.
 - $f(n) = n / \log n$ is smaller than n^1 , but only by a logarithmic factor, **not a polynomial gap** n^e (violates Case 1 condition).

If any equation breaks the strict rules for **a**, **b**, or the polynomial gap condition for **f(n)**, **DO NOT use the Master Theorem**. Fall back to **Recursion Trees** or the **Substitution Method (using mathematical induction)** instead!